



Planlage Kamera - Technische Dokumentation v2.0.7

Diese Dokumentation beschreibt die native Nowofol Kamera-Software, die Auslieferung als Windows-EXE, die Media-Foundation-basierte Kameraanbindung, Einstellungen, Speicherlogik und wichtige Wartungsschritte.

Die Anwendung wurde gemeinsam mit Codex AI erstellt und weiterentwickelt.

Inhaltsverzeichnis

1. Zweck der Software
2. Architekturwechsel in Version 2.0.0
3. Dateien, Auslieferung und Installationspaket
4. Start der fertigen Anwendung
5. Bedienoberfläche
6. Aufnahme und Pflichtfelder
7. Dateiname und Platzhalter
8. Admin-Zugang und Passwortschutz
9. Einstellungen
10. Kamera, Auflösung und Media Foundation
11. Performance des Livebilds
12. Zoom und Referenzlinie
13. Ablage und Explorer-Button
14. Update-System und GitHub Releases
15. Code-Struktur
16. Versionierung, Fehlerbehebung und Pflege

1. Zweck der Software

Die Anwendung zeigt ein Livebild einer angeschlossenen Webcam an. Der Anwender beurteilt darüber die Rolle und speichert bei Bedarf ein Foto.

Vor jeder Aufnahme müssen Auftragsnummer, Rollennummer und Gewicht eingetragen werden. Datum und Uhrzeit werden automatisch vom System erzeugt.

2. Architekturwechsel in Version 2.0.0

Version 2.0.0 ersetzt die bisherige Python/OpenCV-Ausführung durch eine native Windows-Desktop-Anwendung auf Basis von .NET 8 WinForms.

Die Kamera wird über Windows.Media.Capture und MediaFrameReader angebunden. Diese Windows-Kamera-API läuft über den Media-Foundation-basierten Kamerapfad und verwendet die von Windows bereitgestellten Gerätenamen.

Der Wechsel reduziert die Abhängigkeit von Python, verbessert das Verhalten mit hochauflösenden Webcams und macht die Auslieferung als eigenständige Windows-EXE möglich.

3. Dateien, Auslieferung und Installationspaket

dist/Planlage_Kamera.exe: Fertige Windows-Anwendung für den Anwender. Diese Datei kann per Doppelklick gestartet werden.

dist/Ausfuehrliche_Dokumentation.pdf: Technische Dokumentation passend zur EXE.

native/PlanlageKameraNative/Program.cs: Quellcode der nativen Anwendung mit den wichtigsten Klassen und Kommentaren.

native/PlanlageKameraNative/PlanlageKameraNative.csproj: Projektdatei für Build und



Veröffentlichung.

assets/: Nowofol-Logo, App-Icon und Branding-Dateien. Das Logo wird zusätzlich in die native Anwendung eingebettet.

config.json: Wird beim ersten Start automatisch neben der EXE erstellt und enthält Einstellungen sowie den Admin-Passwort-Hash.

Die alte Python/OpenCV-Auslieferung wurde entfernt. Maßgeblich ist die Windows-App aus dist/Planlage_Kamera.exe.

Ein Windows-Installationspaket, zum Beispiel MSIX oder ein klassischer Installer, verbessert nicht direkt die Livebild-Performance. Es ist aber sinnvoll für saubere Installation, Startmenüeintrag, feste Speicherorte, Updates, Berechtigungen und Verteilung auf mehreren PCs.

4. Start der fertigen Anwendung

Die fertige Anwendung befindet sich unter dist/Planlage_Kamera.exe. Für den normalen Einsatz wird diese EXE gestartet.

Die native Release-Veröffentlichung wird als selbständige win-x64-Ausgabe erstellt. Python wird auf dem Ziel-PC nicht benötigt.

Beim ersten Start legt die Anwendung automatisch eine config.json neben der EXE an. Für die Kamera muss Windows den Zugriff auf Kameras erlauben.

5. Bedienoberfläche

Links befindet sich das Livebild der Kamera. Rechts befinden sich Pflichtfelder, Aufnahmebutton, Explorer-Button, Einstellungen, Kameraauflösung, Zoomsteuerung, Referenzlinie und Ablageinformation.

Die Oberfläche nutzt Nowofol-Blau #0A4E88, das Nowofol-Logo und eine modernisierte App-Struktur mit klarer Kopfzeile, ruhigen Flächen und konsistenten Dialogen.

Ab Version 2.0.1 besitzt die Kopfzeile einen Dark-Mode-Toggle. Die Einstellung wird in der config.json gespeichert und beim nächsten Start wiederhergestellt.

Ab Version 2.0.3 wurde die Kopfzeile vergrößert, damit Logo und Toggle nicht mehr abgeschnitten werden. Die Ablageanzeige im Hauptfenster nutzt ein Readonly-Feld mit Tooltip für den vollständigen Pfad.

Ab Version 2.0.4 wurden Logo und Dark-Mode-Toggle erneut ausgerichtet. Der Toggle besitzt nun einen sichtbaren Rand im Dark Mode. Die rechte Bedienleiste nutzt feste Zeilenhöhen, damit Eingabefelder beim Maximieren nicht verzerren.

Die Ablageanzeige wird ab Version 2.0.4 einzeilig über die volle Breite der rechten Bedienleiste dargestellt.

Ab Version 2.0.5 heißt die Anwendung in Fenster und EXE wieder schlicht Planlage Kamera. Das Wort Native wurde aus der sichtbaren Auslieferung entfernt. Logo und Dark-Mode-Toggle wurden erneut vereinfacht und stabil ausgerichtet.

Ab Version 2.0.6 ist die GitHub-Updatequelle standardmäßig auf MarioNowofol/Planlage_Kamera gesetzt.

Ab Version 2.0.7 wird in README und technischer Dokumentation ausgewiesen, dass die Anwendung mit Codex AI erstellt wurde.

Ab Version 2.0.8 verwendet die App standardmäßig den öffentlichen Update-Kanal MarioNowofol/Update_Planlage_Kamera. Bestehende Konfigurationen mit dem alten Standard-Repository werden automatisch migriert.



Das Livebild skaliert mit der Fenstergröße. Die Anwendung vergrößert ihr Fenster nicht selbstständig.

Admin-Dialog und Einstellungen werden mittig über dem Hauptfenster geöffnet.

6. Aufnahme und Pflichtfelder

Auftragsnummer, Rollennummer und Gewicht sind Pflichtfelder. Wenn eines dieser Felder leer ist, wird keine Aufnahme gespeichert.

Nach erfolgreicher Aufnahme werden die Eingabefelder geleert und der gespeicherte Dateiname wird in der Statuszeile angezeigt.

Die Aufnahme wird aus dem zuletzt verfügbaren Kameraframe erzeugt. Bei aktiviertem digitalem Zoom wird der gleiche Zoom-Ausschnitt auch in der gespeicherten JPG-Datei verwendet.

7. Dateiname und Platzhalter

Der Dateiname wird aus einem konfigurierbaren Muster gebildet. Das Standardmuster lautet:

{auftrag}_{rolle}_{gewicht}_{datum}_{uhrzeit}.jpg

Verfügbare Platzhalter: {auftrag}, {rolle}, {gewicht}, {datum}, {uhrzeit}.

Nicht für Windows-Dateinamen geeignete Zeichen und Leerzeichen werden durch Unterstriche ersetzt.

Beispiel: AB123_R12_25kg_20260630_081500.jpg.

Wenn ein Dateiname bereits vorhanden ist, ergänzt die Anwendung automatisch eine laufende Nummer.

8. Admin-Zugang und Passwortschutz

Beim Klick auf Einstellungen fragt die Anwendung zuerst das Admin-Passwort ab. Erst nach korrekter Eingabe wird das Einstellungsfenster geöffnet.

Das Standardpasswort für den Admin-Zugang lautet: admin.

Das Passwort wird in der config.json nicht im Klartext gespeichert, sondern als SHA-512-Hash im Feld AdminPasswordHash.

Das Standardpasswort sollte direkt nach der Inbetriebnahme in den Einstellungen geändert werden.

9. Einstellungen

Im Einstellungsfenster können Ablagepfad, Kameraquelle, Dateinamenmuster, Zoomfunktion und Admin-Passwort angepasst werden.

Ab Version 2.0.3 besitzt der Ablagepfad einen Auswählen-Button, der einen Windows-Ordnerdialog öffnet. Unter dem Dateinamenfeld werden die verfügbaren Platzhalter direkt angezeigt.

Ab Version 2.0.5 können im Einstellungsfenster auch GitHubOwner, GitHubRepo und die automatische Updateprüfung gepflegt werden.

Die Kameraquelle wird mit dem von Windows gemeldeten Gerätenamen angezeigt, damit identische Kamera-IDs besser unterscheidbar sind.

Die Option DarkModeEnabled wird automatisch über den Toggle im Hauptfenster gesetzt und muss normalerweise nicht manuell bearbeitet werden.

Zum Ändern des Passworts wird im Feld Neues Admin-Passwort ein neuer Wert eingetragen.

Bleibt das Feld leer, bleibt das bisherige Passwort erhalten.

Nach dem Speichern werden die Einstellungen in config.json abgelegt. Bei geänderter



Kameraquelle wird die Kamera neu gestartet.

10. Kamera, Auflösung und Media Foundation

Die Kameraanbindung verwendet `Windows.Media.Capture`, `MediaFrameSource` und `MediaFrameReader`. Dieser Pfad nutzt die Windows-Kamera-Infrastruktur und ist Media-Foundation-basiert.

Beim Start wählt die Anwendung aus den von der Kamera gemeldeten Farbkameraquellen und Formaten automatisch die beste preview-freundliche Kombination. Bevorzugt werden BGRA8, NV12, YUY2, MJPEG und RGB32; reine H264-Formate werden nicht mehr bevorzugt.

Ab Version 2.0.2 wird der Startstatus des `MediaFrameReader` geprüft. Wenn Windows den `FrameReader` nicht starten kann, wird ein Fehler angezeigt und nicht mehr stillschweigend ein schwarzes Bild als bereit gemeldet.

Ab Version 2.0.3 wartet die Anwendung nach dem Start auf den ersten verwertbaren Frame. Liefert eine Kamera innerhalb von 3,5 Sekunden kein Livebild, wird automatisch die nächste Kameraquelle getestet.

Die aktive Auflösung wird rechts in der Bedienleiste angezeigt.

Die Kamera muss in Windows freigegeben sein. Blockierte Kamera-Berechtigungen oder ein bereits exklusiv nutzendes anderes Programm verhindern den Start.

11. Performance des Livebilds

Die neue Version verhindert einen Rückstau von 4K-Frames in der Bedienoberfläche. Wenn die UI gerade ein Bild zeichnet, werden Zwischenframes verworfen und nur der neueste Frame weiterverarbeitet.

Die Vorschau wird in einem eigenen Preview-Control gezeichnet und mit bilinearer Interpolation auf die aktuelle Fenstergröße skaliert. Das reduziert CPU-Last gegenüber der alten Python/OpenCV-Vorschau.

Die SoftwareBitmap-Konvertierung berücksichtigt den tatsächlichen Speicher-Stride des Kameraformats. Das ist wichtig für hochauflösende Formate mit Zeilen-Padding.

Ab Version 2.0.1 wird BGRA-Kameramaterial mit ignoriertem Alpha-Kanal in ein sichtbares RGB-Bitmap kopiert. Dadurch werden schwarze Livebilder vermieden, die durch Treiber mit leerem Alpha-Kanal entstehen können.

Ab Version 2.0.2 schreibt die Anwendung eine `camera_diagnostics.log` neben die EXE. Darin stehen erkannte Kameras, gewähltes Format, `FrameReader`-Startstatus und relevante Fehler.

Ab Version 2.0.3 werden auch Direct3D-Kameraframes in SoftwareBitmaps umgewandelt. Dadurch können Kameras angezeigt werden, die keine direkte SoftwareBitmap liefern.

Die Umwandlung von SoftwareBitmap nach GDI+-Bitmap nutzt `CopyToBuffer` statt direktem COM-Speicherzugriff. Das vermeidet WinRT-Projektionsfehler wie `Invalid cast from WinRT.IInspectable`.

Für noch höhere Performance wäre als späterer Ausbauschritt eine Direct3D-basierte Vorschau möglich. Die aktuelle Version setzt bewusst auf eine robuste Windows-Desktop-EXE ohne zusätzliche Bedienkomponenten.

12. Zoom und Referenzlinie

Die Zoomsteuerung kann in den Einstellungen aktiviert oder deaktiviert werden. Ist sie aktiviert, erscheint rechts ein Zoomregler von 1.0x bis 4.0x.

Der digitale Zoom wird mittig aus dem Kamerabild ausgeschnitten. Bei der Speicherung wird derselbe Ausschnitt auf die volle Bildgröße skaliert.



Die waagerechte Referenzlinie kann direkt im Hauptfenster ein- und ausgeblendet werden. Sie wird mittig im Livebild angezeigt und hilft beim Ausrichten der Kamera.
Die Referenzlinie dient nur der Ausrichtung und wird nicht in das gespeicherte Bild geschrieben.

13. Ablage und Explorer-Button

Der Ablagepfad wird in den Einstellungen gesetzt. Beim Speichern wird der Ordner automatisch erstellt, sofern Windows dies erlaubt.

Der Button Ablage öffnen startet den Windows Explorer direkt im aktuellen Ablageordner.
Bei Netzwerklagen muss der Pfad erreichbar sein und der angemeldete Windows-Benutzer benötigt Schreibrechte.

14. Update-System und GitHub Releases

Ab Version 2.0.5 kann die Anwendung beim Start ein GitHub Repository auf neue Releases prüfen. Dazu werden GitHubOwner und GitHubRepo aus den Einstellungen beziehungsweise aus config.json verwendet.

Ab Version 2.0.8 ist der Standard-Updatekanal MarioNowofol/Update_Planlage_Kamera. Dadurch kann das Code-Repository privat bleiben, während Ziel-PCs die öffentlichen Release-Dateien ohne GitHub-Token abrufen können.

Die App ruft die GitHub Releases API ab, vergleicht die neueste Release-Version mit der lokalen App-Version und sucht im Release nach dem Asset Planlage_Kamera.exe.

Wenn eine neuere Version gefunden wird, fragt die App den Anwender. Bei Zustimmung wird die neue EXE in den Temp-Ordner geladen, die laufende App beendet, die EXE ersetzt und anschließend neu gestartet.

Patchnotes werden in PATCHNOTES.md gepflegt und sollten zusätzlich als GitHub Release Notes übernommen werden.

15. Code-Struktur

AppConfig und AppConfigService verwalten config.json, Version, Standardwerte und SHA-512-Passwort-Hash.

MainForm baut die Oberfläche auf, startet die Kamera, validiert Pflichtfelder, speichert Bilder und öffnet Einstellungen.

MediaFoundationCameraService kapselt Geräteauflistung, Kamera-Start, Formatwahl und MediaFrameReader-Verarbeitung.

PreviewControl zeichnet Livebild, Zoomausschnitt und Referenzlinie.

SoftwareBitmapConverter kopiert Kamera-Frames aus dem WinRT-Speicher in GDI+-Bitmaps.

ImageTools und FileNameBuilder enthalten gemeinsame Bild- und Dateinamenlogik.

16. Versionierung, Fehlerbehebung und Pflege

Die aktuelle Version der Anwendung lautet 2.0.8. Die Versionsnummer wird in der Titelleiste und im Kopfbereich der Anwendung angezeigt.

Bei künftigen Funktionsänderungen wird die Versionsnummer erhöht und diese Dokumentation angepasst.

EXE startet nicht: Datei lokal speichern, Windows-Sicherheitsmeldung prüfen und erneut starten.

Keine Kamera gefunden: Kamera anschließen, andere Programme schließen, Treiber und



Windows-Kameraberechtigung prüfen.

Livebild ruckelt weiterhin: USB-3-Anschluss verwenden, andere Kamera-Programme schließen und Kamera-Treiber prüfen. Manche 4K-Webcams liefern die höchste Auflösung nur mit reduzierter Bildrate.

Ablagepfad kann nicht erstellt werden: Pfad und Schreibrechte prüfen.

Falsches Admin-Passwort: Mit dem zuletzt gespeicherten Admin-Passwort anmelden. Bei Erstinbetriebnahme lautet es admin.

Die erzeugten JPG-Dateien sollten nach den internen Vorgaben gesichert werden. Änderungen an config.json sollten im Normalfall nur über den Admin-Bereich erfolgen.